Lecture 4 - Widgets & Layouts

Material, Lists, Forms - Calculator / Function-Plotter UI

Fettah KIRAN

Department of AI and Data Engineering

Yıldız Technical University

fkiran@yildiz.edu.tr

Learning Outcomes

- Build Material screens with `Scaffold`, `AppBar`, rows/columns/stacks
- Explain Stateless vs Stateful + lifecycle and use `setState` correctly
- Render long lists efficiently with builders + keys
- Validate forms with `GlobalKey<FormState>` for math inputs
- Debug constraints, use `const`, DevTools, and responsive patterns

Source: https://docs.flutter.dev/learn/pathway

Overview

1. Recap Quiz: Week 3 (abstract, mixin, generics, Future/await)
2. Everything is a Widget + Widget Tree
3. Stateless vs Stateful + Lifecycle
4. MaterialApp / Scaffold / AppBar
5. Layouts: Row, Column, Stack, Expanded, Container
6. Constraints, Lists/Grids, Keys, Assets, Themes
7. Text, Forms, Buttons, State Lifting, const & Performance
8. Responsive, Errors, Math Focus, Lab: Calculator / Plotter UI

Readings & Resources for Week 4.

In this lecture, we covered the related topics of **Widgets** and **Layouts** in Flutter. We explored the following key concepts from the official Flutter documentation:

Widgets: https://docs.flutter.dev/ui/widgets
Layouts: https://docs.flutter.dev/ui/layout
CLI commands: https://docs.flutter.dev/reference/flutter-cli

1. Everything is a Widget

- UI = tree of immutable widget configs; Flutter inflates to Elements + RenderObjects
- Composition over inheritance: small widgets nest into screens
- Two families: **stateless** (static) vs **stateful** (dynamic)
- Layout, painting, gestures - all widgets: `Padding`, `Center`, `GestureDetector`

```
MaterialApp > Scaffold > Column > [Text, Row, ElevatedButton]
```

Widget Tree - Mental Model

- **Widget:** cheap blueprint, rebuilt often - `Text('x=2')`
- **Element:** stable instance in tree, holds state slot
- **RenderObject:** layout + paint, expensive - cached
- Rebuild = new widgets diffed against elements; only dirty subtrees repaint
- Deep trees are fine; *wide rebuilds* are not - split into small widgets

> Rule: `build()` must be pure - no network, no random, no side effects.

StatelessWidget

- Immutable fields (`final`), renders from inputs only
- Use for labels, icons, static cards, plot legends
- Example: `ResultLabel(value)` returns `Text('f(x) = ...')` - parent rebuilds it
- No `setState`; same inputs → same UI

StatefulWidget + setState - Code 1/10

```
class CalcDisplay extends StatefulWidget {
  const CalcDisplay({super.key});
  @override
  State<CalcDisplay> createState() => _CalcDisplayState();
}
class _CalcDisplayState extends State<CalcDisplay> {
  String expr = '';
  void append(String s) => setState(() => expr += s);
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(expr, style: const TextStyle(fontSize: 32)),
      ElevatedButton(onPressed: () => append('7'), child: const Text('7')),
    ]);
  }
}
```

- `setState()` marks element dirty → `build()` re-runs. Never call in `build()` itself.

Lifecycle: initState / build / dispose

- `initState()` - once: create `TextEditingController`, subscribe, set initial range
- `build()` - many times: pure, fast, no I/O
- `dispose()` - once: `controller.dispose()`, cancel timers/streams - **forget = leak**
- `didUpdateWidget` / `didChangeDependencies` - advanced: react to new props/theme

```
initState → build → (setState → build)* → dispose
```

> Plotter: init `min/max/step` controllers in `initState`, dispose them.

MaterialApp / Scaffold / AppBar - Code 2/10

https://docs.flutter.dev/ui/widgets/material

https://m3.material.io

```
MaterialApp(
  title: 'Math UI', theme: ThemeData(useMaterial3: true),
  home: Scaffold(
    appBar: AppBar(title: const Text('Calculator')),
    body: const CalcBody(),
    floatingActionButton: FloatingActionButton(
      onPressed: () {}, child: const Icon(Icons.calculate)),
  ),
);
```

- `MaterialApp` = theme + navigation + locale shell. One per app.
- `Scaffold` = screen skeleton: appBar, body, FAB, drawer, snackBar.

Row / Column Basics

- `Row`: horizontal; `Column`: vertical. Main vs cross axis.
- `mainAxisAlignment`: start, center, spaceBetween, spaceEvenly
- `crossAxisAlignment`: center, stretch, start
- `mainAxisSize.min` - shrink-wrap; default `max` fills
- Nest rows in columns for keypads; wrap in `SingleChildScrollView` if overflow risk

> Keypad row: `Row(children: ['7','8','9','÷'].map(btn).toList())`

Stack / Container / Padding / Center

- `Stack` - overlay: plot canvas + floating label / legend badge (`Positioned`)
- `Container` - padding + margin + decoration + size (convenience, composes others)
- `Padding` - insets: `EdgeInsets.all(16)`, `.symmetric`, `.only`
- `Center` - center child; `Align` for custom alignment; `SizedBox` for gaps

Prefer `Padding` + `DecoratedBox` over heavy `Container` when only one feature needed.

Expanded / Flexible - Code 3/10

```
Row(children: const [
  Expanded(flex: 2, child: ExprField()), // takes 2x space
  SizedBox(width: 8),
  Expanded(child: EqualsButton()),       // takes 1x space
])
```

- `Expanded` = `Flexible(fit: FlexFit.tight)` - forces fill. `Flexible.loose` lets child shrink.
- Calculator keys: each `Expanded` in `Row` → equal-width buttons on any screen.
- Never put `Expanded` directly in `SingleChildScrollView` / unbounded parent - crash.

Constraints - Code 4/10

```
Container(
  constraints: const BoxConstraints(minH: 48, maxW: 400),
  padding: const EdgeInsets.all(12),
  child: const Text('min=0  max=10  step=0.5'),
)
```

- Rule: **constraints go down, sizes go up, parent positions.**
- `BoxConstraints`: min/max width/height; `tight` = fixed, `loose` = up-to.
- `MediaQuery.sizeOf(context)` for screen-aware widths; `LayoutBuilder` for parent-aware.

Overflow Clinic - Fix It!

- Yellow-black stripes = overflow by N px. Causes: fixed width + keyboard + long `Row`.
- Fixes: wrap in `Expanded/Flexible`, `SingleChildScrollView`, `Wrap`, shrink text with `FittedBox` / `Expanded`.
- **Task:** `Row(Text(longExpr) + Button)` overflows → which widget gets `Expanded`? Why not the button?

- Open DevTools → *Layout Explorer* to see constraints live.

ListView.builder - Code 5/10

```
ListView.builder(
  itemCount: history.length,
  itemBuilder: (c, i) {
    final h = history[i];
    return ListTile(
      leading: const Icon(Icons.history),
      title: Text(h.expr),
      subtitle: Text('= ${h.result}'),
      trailing: IconButton(icon: const Icon(Icons.delete),
        onPressed: () => removeAt(i)),
    );
  },
)
```

- Lazy: builds only visible rows - 10k plot-history entries OK.
- `ListView.separated` adds dividers; `shrinkWrap` only inside another scrollable.

GridView + Keys

- `GridView.builder(gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 4))` → calculator keypad
- `crossAxisSpacing/mainAxisSpacing` + `childAspectRatio` tune key shape
- **Keys:** `ValueKey(h.id)` preserves identity on delete/reorder - without keys, wrong row deletes
- Dismissible history: `Dismissible(key: ValueKey(id), onDismissed: ...)` for swipe-to-delete

Assets & Images - Code 6/10

```
# pubspec.yaml
flutter:
  assets:
    - assets/ytu-logo.png
    - assets/plots/
```

```
Image.asset('assets/ytu-logo.png', height: 48);
Image.network(url, loadingBuilder: (_, w, p) =>
  p == null ? w : const CircularProgressIndicator());
```

- Declare folder, `flutter pub get`, hot-restart after adding files.
- Math: bundle formula sheets / logo; network: fetch chart thumbnails with placeholder + errorBuilder.

Themes - Code 7/10

```
MaterialApp(
  theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.indigo),
  darkTheme: ThemeData.dark(useMaterial3: true),
  themeMode: ThemeMode.system, // or .light / .dark
);
```

- `Theme.of(context).colorScheme.primary` - never hard-code hex in widgets.
- `TextTheme`, `CardTheme`, `InputDecorationTheme` centralize plotter form style.
- Week 12 localization: theme + locale both via `MaterialApp`.

Dark Mode Toggle

- Store `ThemeMode` in state (later: Provider); toggle via `SwitchListTile` / IconButton
- Test both: expression field contrast, grid lines visible on dark canvas
- Use `colorScheme.surface/onSurface`, not `Colors.white/black`
- SnackBar confirms: `ScaffoldMessenger.of(context).showSnackBar(...)`

> - **Task:** switch emulator to dark - which hard-coded color breaks first in your lab UI?

Text / TextField Controllers

- `Text` styles: `TextStyle(fontSize, fontWeight, fontFeatures)`; `SelectableText` for copyable results
- `TextField`: `controller`, `keyboardType: TextInputType.numberWithOptions(decimal: true)`, `decoration: InputDecoration(labelText: 'f(x)')`
- Controllers are state - init in `initState`, dispose in `dispose`
- `onChanged` for live plot preview; `onSubmitted` for equals action

Form + Validation - Code 8/10

```
final key = GlobalKey<FormState>();
double? min, max, step;
Form(key: key, child: Column(children: [
  TextFormField(decoration: const InputDecoration(labelText: 'min'),
    keyboardType: TextInputType.number,
    validator: (v) => v==null||v.isEmpty ? 'Required' : null,
    onSaved: (v) => min = double.tryParse(v!)),
  TextFormField(decoration: const InputDecoration(labelText: 'max'),
    validator: (v) {
      if (v==null||v.isEmpty) return 'Required';
      if ((double.tryParse(v) ?? 0) <= (min ?? 0)) return 'max > min';
      return null;
    }),
  ElevatedButton(onPressed: () {
    if (key.currentState!.validate()) { key.currentState!.save(); plot(); }
  }, child: const Text('Plot')),
]));
```

- Expression field: reject empty + illegal chars via `RegExp(r'^[0-9+\-*/(). ]+$')`.

Buttons

- `ElevatedButton` - primary (= / Plot); `FilledButton` (M3), `TextButton` - secondary (C / Clear); `IconButton` - compact (⌫, history)
- `FloatingActionButton` - calculate action; `SegmentedButton` - rad/deg, 2D/3D toggle
- Style once: `ElevatedButton.styleFrom(minimumSize: Size(64,56))` for fat calculator keys
- Disable during solving: `onPressed: solving ? null : solve`

State Lifting

- Single source of truth lives in nearest common ancestor
- `CalcScreen` owns `expr`; `Keypad(onKey)` + `Display(expr)` are dumb children
- Callbacks down (`void Function(String)`), state up - same pattern Week 11 MVVM scales with Provider

```
CalcScreen (expr, history)
  ├─ Display(expr)
  └─ Keypad(onKey: append)
```

- Plotter: form owns min/max/step; passes `onPlot(min,max,step)` up.

const Correctness - Code 9/10

```
const Padding(
  padding: EdgeInsets.all(16),
  child: Text('π ≈ 3.14159'), // const subtree skips rebuild
)
```

- `const` widgets canonicalized - rebuild cost ~zero. Lint: `prefer_const_constructors`.
- Hot Reload keeps state; Hot Restart resets - know which you pressed when state "sticks".
- Split big `build()` into small `const`-friendly widgets: `KeyButton`, `HistoryTile`.

Performance Tips

- Builders, not giant `children: [...]` lists - lazy by default
- `const`, `RepaintBoundary` around animated plot canvas, avoid `Opacity` in scrolls
- Keys only where identity matters (history, dismissibles) - over-keying hurts
- Profile: DevTools → *Rebuild stats* - find widgets rebuilding 60×/s

> Math app killer: rebuilding whole history on each keystroke - lift `TextField` state down.

DevTools - Widget Inspector

- Open: `F1 → Dart: Open DevTools` or `flutter pub global run devtools`
- **Select Widget Mode:** tap emulator pixel → jumps to source line
- Layout Explorer: constraints, flex factors, overflow flags
- Rebuild rainbow: highlights wasteful rebuilds live

> - **Task:** inspect your keypad `Row` - what `flex` does each key have? Force an overflow, screenshot stripes.

Responsive - Code 10/10

```
LayoutBuilder(builder: (c, box) {
  final wide = box.maxWidth > 600;
  return wide
    ? Row(children: const [Expanded(child: PlotCanvas()),
                           Expanded(child: PlotForm())])
    : Column(children: const [PlotCanvas(), PlotForm()]);
});
```

- `MediaQuery.sizeOf(context)` - orientation, keyboard (`viewInsets.bottom`), textScale
- `OrientationBuilder` / `LayoutBuilder` - side-by-side plot on tablet, stacked on phone
- Test: Chrome device toolbar 360px → 1024px; font scale 1.3× must not overflow.

Errors & FutureBuilder Preview

- `FutureBuilder<double>(future: solve(expr), builder: (c,s) { if (s.hasError) return ErrorCard(...); if (!s.hasData) return CircularProgressIndicator(); return Text('= ${s.data}'); })`
- Never `try/catch` inside `build` - surface error widgets: `Icon(Icons.error) + Retry`
- Empty states: history empty → `Center(Text('No computations yet'))`, not blank screen
- Week 6/9 deep-dive: HTTP + Firestore streams reuse this pattern.

Math Focus: Calculator Keypad

- `GridView.count(crossAxisCount: 4)` keys: `7 8 9 ÷ / 4 5 6 × / 1 2 3 − / 0 . = +`
- Display: right-aligned `Text(expr)`, `FittedBox` to shrink long expressions
- Expression `TextFormField` validates `RegExp` before `solve()` from Week 3
- Clear (`C`) vs backspace (`⌫ IconButton`) vs `=` (`ElevatedButton`, flex 2?)

Math Focus: Plotter Form

- Three `TextFormField`s: min / max / step + function `f(x)` field
- Validators: required, numeric, `max > min`, `step > 0 && step < (max−min)`
- `keyboardType: numberWithOptions(decimal:true, signed:true)` - minus matters!
- On Plot: `validate() → save() → generate points → push to history + canvas`

Math Focus: History List

- Model: `class Entry { final String id, expr; final double result; }`
- `ListView.builder` + `ValueKey(id)` + `Dismissible` swipe-delete
- Tap entry → refill expression field (recall); long-press → copy result
- Persist later (Weeks 8–9: Hive/Firestore); today in-memory `List<Entry>`

Live Demo (Instructor)

- Keypad `GridView` + expression validation → intentional error (divide by zero → error card)
- Toggle dark mode; rotate emulator - responsive Row→Column
- Inspector: select a key, show `Expanded` flex + rebuild highlight
- Your turn in lab: replicate display + 4 keys in 10 min.

Lab - Overview

- **Goal:** calculator + function-plotter *UI* (no solver logic - stub results, wire Week 3 later)
- Pairs, one laptop drives, other inspects. `flutter create math_ui` → `lib/main.dart`
- Deliver: keypad grid, validated expression field, min/max/step form, history list
- Timebox: 60 min build + 10 min show-and-tell. Stub `solve()` returns `42.0`.

Lab - Calculator UI Steps

1. `Scaffold(AppBar: 'Calculator')` + display `Text` (right-aligned, fontSize 36)
2. Expression `TextFormField` + `RegExp` validator + `C` / `⌫` buttons
3. `GridView.count(4)` keys `0–9 . + − × ÷ =` - each `Expanded`-like via grid delegate
4. `=` validates → `setState` appends `Entry` to history → SnackBar confirms

Lab - Plotter Form Steps

1. `Form(GlobalKey)` with min / max / step + `f(x)` fields, number keyboards
2. Validators: required + `max>min` + `step>0`; `onSaved` parses `double.tryParse`
3. Plot button: `validate() → save()` → stub canvas `Container(height:200, child: Text('plot n points'))`
4. Side-by-side on wide via `LayoutBuilder` (Row) else Column

Lab - History + Polish Steps

1. `ListView.builder` history with `ValueKey` + `Dismissible` + empty-state text
2. `ThemeData(colorSchemeSeed)` + dark-mode toggle `SwitchListTile`
3. `const` pass: add `const` where lint suggests; split `KeyButton` widget
4. Rotate + dark + keyboard-open checks - no overflow stripes

Best Practices

- Small widgets, `const` constructors, lifted state, validated forms
- No logic in `build()` - solvers live in Week 3 classes, UI only displays
- Theme colors via `colorScheme`, spacing via `8pt` multiples
- Name routes/screens for Week 5 navigation: `CalculatorScreen`, `PlotterScreen`

Exit Ticket (3 min)

- On paper / chat: **one** constraint rule + **one** validator rule from today.
- Example: "constraints go down" + "max must exceed min".
- Collect → seeds Week 5 navigation recap quiz. Fastest correct row demos next week.

Resources

- **Flutter Widget Catalog** - every widget with live diagram
- **Layouts Codelab: Basic Flutter layout** - Row/Column/Stack drills
- Practice more with following tutorials:
  - https://docs.flutter.dev/learn/pathway/tutorial/widget-fundamentals
  - https://docs.flutter.dev/learn/pathway/tutorial/layout
- DevTools Inspector + `flutter analyze` - run before every commit

Roadmap - Week 5 Preview

- **Navigation:** `Navigator.push`, named routes, `go_router`
- Lab: multi-screen math app - calculator ⇄ plotter ⇄ history with arguments
- Keep `CalculatorScreen` / `PlotterScreen` split - routes plug in cleanly

Questions & Discussion

- Overflow you hit today? Validator edge case (`step=0` , empty expr)?
- Team repo set to private + member commits visible? Fix before Week 10 milestone.

Thank you!

Build the keypad, validate the math, list the history - see you Week 5 for Navigation!